

Design: Transaction Ledger

Record a purchase — ticker, quantity, price paid per unit — and let the app keep quantity and average price correct on its own, instead of the user doing the arithmetic by hand and typing the result into two cells.

Status: proposed · Author: design pass, 2026-09-05 · Scope: core/
+ TUI + CLI

§1 Why

Today the spreadsheet stores only the *result* of your buying history:

TICKER	QUANTITY	EXCHANGE	CATEGORY	AVERAGE PRICE
ITSA4.SA	9105	B3	Stock	7.84

When you buy 100 more shares at R\$14.20 you have to work out the new weighted average (R\$7.9092), then edit two cells — quantity with **u**, average with **P**. Get it wrong and every P&L number in the app is quietly wrong, with no record of how the position was built.

A transaction ledger fixes both halves: the app does the arithmetic, and the history that produced the numbers is written down.

Goals

- Record a buy with three inputs: **ticker, quantity, price per unit**.
- Recompute that ticker's quantity and average price automatically.
- Keep an auditable history of how each position was built.
- Change nothing about how the engine, fetcher, or existing UI work.
- Work on an existing spreadsheet with no migration step from the user.

Non-goals (v1)

- Sells and realised P&L — the schema reserves room for them (§4, §12).
- Brokerage fees and taxes.
- Editing or deleting a recorded transaction.
- Multi-currency conversion at transaction time — see the currency rule in §6.

§2 What the user does

In the TUI

Portfolio tab press T

Add transaction

Ticker

ITSA4.SA

Quantity

100

Price per unit (BRL)

14.20

Date (optional, default today)

2026-09-05

[Record] [Cancel]

← prefilled from

the selected row

← currency comes

from \$6

toast: ITSA4.SA +100 @ 14.20 → 9,205 @ 7.9092

T is free — the existing bindings are `q e c h escape p d w n v u/U P A D`.

Uppercase matches the convention already in `app.py:110`: mutating actions take Shift, filters and sorts are lowercase.

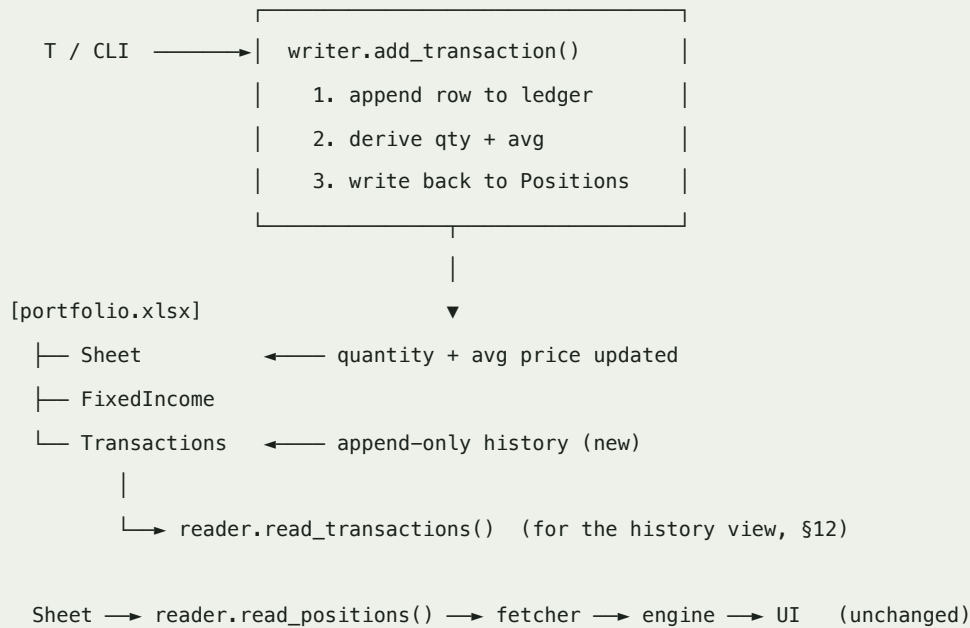
From the CLI

```
portfolio add-transaction ITSA4.SA 100 14.20
portfolio add-transaction SAP.DE 5 185.40 --date 2026-09-01 --note "monthly buy"
portfolio add-transaction ITSA4.SA 100 14.20 --dry-run    # show, write nothing
```

Same subcommand shape as the existing `portfolio import (__main__.py:34)`.

§3 Where it fits

The ledger sits *behind* the positions sheet. Nothing downstream changes, because the positions sheet stays the contract the engine reads:



This is the single most important structural decision in the design: **the ledger is written, then folded into the positions sheet in the same operation.** The engine, the fetcher, `PositionValue`, every widget, and every existing test keep working without modification.

§4 Storage

A new sheet named `Transactions`, created on demand, in the same workbook. Append-only. Row 1 is a header, matching how `Sheet` and `FixedIncome` are laid out today.

COL	FIELD	TYPE	REQUIRED	NOTES
A	Date	date	yes	Trade date. Defaults to today.
B	Ticker	text	yes	Exactly as it appears in the positions sheet.
C	Side	text	yes	BUY. SELL is reserved and rejected in v1.
D	Quantity	number	yes	Units bought, > 0.
E	Price	number	yes	Per unit, in the currency from §6, >= 0.
F	Note	text	no	Free text. Used by the opening-balance row (§5).

Example after recording one buy against an existing holding:

	A	B	C	D	E	F
1	Date	Ticker	Side	Qty	Price	Note
2	2026-09-05	ITSA4.SA	BUY	9105	7.84	opening balance (from positions sheet)
3	2026-09-05	ITSA4.SA	BUY	100	14.20	

Why a Side column in v1 when only BUY is written. Adding a column to a spreadsheet people already have data in is the expensive kind of change — every existing row needs backfilling and the reader needs a version check. Reserving the column now costs one constant; adding it later costs a migration.

Why a separate sheet rather than a separate file. One file stays one backup, one path in `settings.excel_path`, and one thing to open in Excel. The reader already treats a missing sheet as "return an empty list" (`reader.py:130`), so old workbooks keep working untouched.

§5 The arithmetic

Weighted average cost

For a ticker's buy rows:

```
quantity = Σ qty
cost      = Σ (qty × price)
average   = cost / quantity
```

Worked example — the ITSA4 case from §1:

```
opening  9105 × 7.84   =  71,383.20
buy      100 × 14.20  =   1,420.00
                                     _____
                        9205      72,803.20  →  avg = 7.90909...
```

The opening-balance row

Here is the problem this solves. Your spreadsheet already says ITSA4.SA is 9105 shares at R\$7.84, and no transaction produced that — it predates the feature. If the ledger is authoritative, recording your first 100-share buy would drop the position to 100 shares. If the ledger is only additive, it can never be trusted as history.

Three options were considered:

APPROACH	VERDICT
A Ledger is authoritative; user backfills their history by hand	Correct, but demands a data-entry project before the feature is usable
B Add to the existing quantity; ledger is an audit log only	No backfill, but the ledger never explains the whole position
C On the first transaction for a ticker, snapshot the current sheet values as an opening-balance row, then treat the ledger as authoritative	Chosen

C gives you a ledger that always sums to the position, with no work on your part. The opening row is marked in the `Note` column so it reads honestly as "this is where we started measuring", not as a real trade.

If the ticker has a quantity but **no** average price, the opening row is written with a blank price and the note `opening balance (cost unknown)`. The derived average is then computed over the priced rows only, and the app keeps showing the same partial-cost P&L it shows today — no worse, no invented number.

Sells (reserved, not implemented)

Average-cost method, for when §12 phase 4 lands: a sell reduces `quantity` and leaves `average` untouched; realised P&L is $(\text{sell_price} - \text{average}) \times \text{qty}$. The derivation function is written to this rule now so adding sells is a UI change, not a maths change.

§6 Which currency is the price in?

This is the sharp edge of the feature. The `Average Price` column is **not** in one currency — `engine.py:120` compares it against the live price like this:

ASSET	COLUMN HOLDS	BECAUSE
ITSA4.SA	BRL	native currency of a B3 listing
SAP.DE	EUR	native currency of a Frankfurt listing
BTC-USD	BRL	crypto is bought on a BRL exchange; the engine converts the live USD price to BRL before comparing

A transaction price is stored in exactly the same currency as that column, so the existing P&L maths keeps working with no changes. The rule:

```
def transaction_currency(position) -> str:
    if position.category.lower() == "crypto":
        return "BRL"          # matches engine.py's crypto branch
    return native_currency    # from the snapshot, or inferred from the suffix
```

The UI must *say* which currency it wants — the modal label reads `Price per unit (EUR)`, taken from the last snapshot's `native_currency` for that ticker, falling back to suffix inference (`.SA` → BRL, `.DE` → EUR, `-USD` → BRL via the crypto rule) when the ticker is new. Getting this wrong is silent and expensive: a EUR price typed as BRL corrupts the average with no error message.

§7 Code changes

7.1 `core/models.py`

```
class Transaction(BaseModel):
    """One recorded trade. The input model – no derived values live here."""
    date: date
    ticker: str
    side: Literal["BUY", "SELL"] = "BUY"
    quantity: float = Field(gt=0)
    price_native: float = Field(ge=0)  # ge=0: a bonus share can cost nothing
    note: str = ""

class DerivedHolding(BaseModel):
    """What a ticker's ledger adds up to – the output of core/transactions.py."""
    ticker: str
    quantity: float
    avg_price_native: float | None  # None when no row carries a price
    total_cost: float
    priced_quantity: float  # qty whose cost is actually known
```

7.2 `core/transactions.py` (new)

Pure functions, no I/O, no `openpyxl` – the easiest module in the codebase to test and the one that holds the rules from §5.

```
def derive_holding(ticker: str, transactions: Sequence[Transaction]) -> DerivedHolding
def transactions_for(ticker: str, transactions: Sequence[Transaction]) -> list[Transaction]
```

7.3 `core/reader.py`

```
async def read_transactions(path: Path) -> list[Transaction]
def _parse_transactions(path: Path) -> list[Transaction]  # blocking, in to_thread
```

Mirrors `_parse_fixed_income` exactly: missing sheet → `[]` and a log line; blank rows skipped; anything malformed wrapped in `ExcelParseError`.

7.4 `core/writer.py`

```
async def add_transaction(path: Path, txn: Transaction) -> DerivedHolding
```

One blocking `_add_transaction` behind one `asyncio.to_thread`, opening and saving the workbook exactly once so a single write covers the whole operation:

1. Load workbook (`writable, data_only=False` – same as `_open_positions_sheet`).

2. Create the `Transactions` sheet with its header if absent.
3. If the ledger has no row for this ticker **and** the positions sheet has a quantity for it → append the opening-balance row (§5).
4. Append the user's row.
5. Read every ledger row for the ticker; `derive_holding(...)`.
6. Write the derived quantity to column B and average to column E of the positions sheet. Create the position row if the ticker is new (§8).
7. Save once. Log `transaction_recorded` with ticker, qty, price, derived values.

New column constants alongside the existing ones (`writer.py:38`): `_TXN_COL_DATE =`
`1 ... _TXN_COL_NOTE = 6, _TXN_SHEET = "Transactions".`

7.5 `core/exceptions.py`

```
class TransactionError(Exception):
    """Raised when a transaction cannot be recorded (bad side, unusable ledger)."""
```

Quantity and price violations surface as Pydantic `ValidationError` from the model, exactly as `AddPositionModal` already handles them (`modals.py:205`).

7.6 `ui/tui/modals.py`

```
class AddTransactionModal(ModalScreen[Transaction | None]):
    def __init__(self, ticker: str = "", currency: str = "") -> None
```

Same shape as `AddPositionModal`: labelled inputs, Enter or the button saves, Escape cancels, Pydantic validates before `dismiss()`. The currency from §6 goes in the price label.

7.7 `ui/tui/app.py`

- `("T", "add_transaction", "Transaction")` in `BINDINGS`.
- `action_add_transaction()` — prefills the ticker from `PortfolioTable.selected_ticker()` (empty is fine, unlike the edit actions which require a selection), resolves the currency, pushes the modal.
- `async _apply_add_transaction(txn)` — calls the writer, notifies with the derived result, sets `refresh_event` so the engine re-reads within a second. Identical error handling to the other four `_apply_*` methods.

7.8 `ui/cli/transaction_command.py (new)` + `__main__.py`

`run(ticker, quantity, price, date, note, portfolio, dry_run) -> int`,
 returning a process exit code, matching `import_command.run`. `--dry-run` prints the before/after holding and writes nothing.

§8 Unknown tickers

If the ticker has no row in the positions sheet, the transaction creates it rather than failing — buying something new is the most ordinary reason to record a transaction. Exchange and category are inferred from the ticker suffix, the same rule the summary cards use (`widgets.py:_on_exchange`):

SUFFIX	EXCHANGE	CATEGORY
.SA	B3	(blank)
.DE	Frankfurt	(blank)
-USD	Crypto	Crypto
anything else	(blank)	(blank)

Category is left blank except for crypto because the suffix genuinely doesn't say whether `KNRI11.SA` is a Fund, an ETF, or a Stock — and a wrong category changes both the allocation chart and the crypto currency rule in §6. The toast says what was created (`Created SAP.DE (Frankfurt) — set a category with A`) so the guess is visible rather than silent.

§9 Validation and failure modes

SITUATION	BEHAVIOUR
Quantity ≤ 0 or not a number	Modal/CLI error before any file access (Pydantic <code>gt=0</code>)
Price < 0	Rejected; 0 allowed for bonus/subscription shares
Blank ticker	Rejected
side="SELL" in v1	<code>TransactionError("Sells are not supported yet")</code>
Date in the future	Allowed, with a warning toast — settlement dates are legitimate
Ticker unknown	Position created, §8
Transactions sheet missing	Created with its header
Ledger row unparseable (text in the quantity cell)	<code>ExcelParseError</code> naming the row number; nothing is written
Workbook open in Excel (Windows lock)	<code>PermissionError</code> surfaced as a toast; the ledger is unchanged

§10 Risks

A save is not atomic. `wb.save(path)` truncates and rewrites; the engine reads the same file every 30 s. A read landing mid-save fails with `ExcelParseError`. This risk exists today for all four `_apply_*` edit paths, so the ledger doesn't add it — but the

ledger makes writes more frequent. Suggested hardening, worth doing in the same phase: save to `path.with_suffix(".tmp")` then `os.replace(tmp, path)`, which is atomic on macOS and Linux, and copy to `.bak` before the first write of a session.

Drift between ledger and positions sheet. The quantity and average cells stay editable with `u` and `P`, so a hand edit can disagree with what the ledger sums to. Mitigations, in order of cost: the derived values are rewritten on every transaction (so drift self-heals on the next buy); a future `portfolio ledger check` reports mismatches; making the cells read-only once a ticker has a ledger is the strict option, and is probably too rigid for a tool that is also a spreadsheet.

Float money. Costs accumulate in `float`, so $0.1 + 0.2 \neq 0.3$ exactly. The relative error is $\sim 1e-16$ — invisible next to a stock price, and the codebase is `float` throughout.

`Decimal` is the textbook answer and is worth knowing about; adopting it here would mean converting the whole pipeline, which buys nothing at this scale.

§11 Testing

Following the existing patterns — `tmp_path` workbooks in `test_writer.py`, `Pilot` for modals in `test_modals.py`.

`tests/unit/test_transactions.py` (pure maths, no files) - single buy → quantity and average equal that buy - two buys → weighted average, not the arithmetic mean of the two prices - the §5 worked example: $9105 @ 7.84 + 100 @ 14.20 \rightarrow 9205 @ 7.9092$ - a row with no price → excluded from the average, included in `quantity` - no priced rows at all → `avg_price_native` is `None` - empty ledger → quantity 0, average `None`

`tests/unit/test_reader_transactions.py` - missing sheet → `[]` - blank trailing rows skipped - dates read back as `date`, not `datetime` or text

`tests/unit/test_writer_transactions.py` - sheet created on first transaction, header written once - opening-balance row written for a ticker with existing quantity — and only once - no opening row for a ticker that already has ledger history - positions sheet quantity and average updated to the derived values - unknown ticker → position row created with inferred exchange - `FixedIncome` sheet and untouched positions rows preserved - `SELL` → `TransactionError`, nothing written

`tests/unit/test_transaction_modal.py` - valid input dismisses with a `Transaction` - quantity `0`, negative price, blank ticker → error, dialog stays open - Escape → `None`

`tests/integration/test_transaction_roundtrip.py` - record → `read_positions` → engine snapshot shows the new quantity and P&L computed from the derived average

§12 Phasing

PHASE	CONTENTS	WHY THIS ORDER
1	models, transactions.py, reader, writer, tests, CLI subcommand	The whole feature works headless and is fully testable before any UI exists
2	AddTransactionModal, T binding, _apply_add_transaction	The three-field flow from §2
3	Transactions tab (read-only history, newest first) + "Invested capital" on the Summary	Makes the ledger visible; needs only read_transactions
4	Sells, realised P&L, fees	Needs a Side selector and a realised-P&L column; the schema and maths are already shaped for it (§5)

Phase 1 is the useful unit — after it you can record buys from the terminal and the TUI shows correct numbers on its next refresh.

§13 Open questions

1. **Sells** — phase 4, or needed sooner? It changes the modal (a side selector) more than it changes the core.
2. **Fees** — worth a column now while the schema is being created? Brokerage on B3 is small but real, and it belongs in the average cost if you want a true break-even price.
3. **Backdating** — should a transaction dated last month be allowed to reorder ledger rows, or always append at the end? Append-only is simpler and the derivation is order-independent; the ledger just wouldn't read chronologically.
4. **Quantity editing** — keep **u** and **P** enabled once a ticker has a ledger (drift, §10), or make the ledger the only way to change those numbers?